Web Dev Mastery

📘 Topics Covered in This CSS Grid Masterclass

1. What is CSS Grid?
2. Basic Grid Setup (`display: grid`)
3. Grid Columns and Rows (`grid-template-columns`, `grid-template-rows`)
4. Gap (`grid-gap`, `row-gap`, `column-gap`)
5. Grid Lines & Placement (`grid-column`, `grid-row`)
6. Grid Area Naming (`grid-template-areas`)
7. Implicit vs Explicit Grid
8. Grid Auto Flow (`grid-auto-flow`)
9. Nested Grids
10. Responsive Design with Grid
11. Grid vs Flexbox – When to Use What?

🔹 1. What is CSS Grid?

✅ Use When:

- You need to create **2-dimensional layouts** (both rows & columns).
- You want **precise control** over layout, positioning, and spacing.

🧠 Explanation:

CSS Grid is a layout system for creating web designs by dividing space into columns and rows.

🔹 2. Basic Grid Setup

📝 HTML (`basic-grid.html`)

html

```
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="basic-grid.css">
</head>
<body>
  <div class="grid-container">
    <div>Item 1</div>
    <div>Item 2</div>
    <div>Item 3</div>
    <div>Item 4</div>
  </div>
</body>
</html>
```

🎨 CSS (`basic-grid.css`)

css

```
.grid-container {
  display: grid;
  grid-template-columns: auto auto;
  gap: 10px;
  border: 2px solid #333;
}

.grid-container > div {
  background: #89CFF0;
  padding: 20px;
  text-align: center;
```

```
}
```

📌 Output:

A simple 2-column grid with 4 items.

🔹 3. Grid Columns & Rows

🎨 CSS Example

css

```
.grid-container {
  display: grid;
  grid-template-columns: 100px 200px 1fr;
  grid-template-rows: 50px 100px;
}
```

- `1fr`: fraction of available space.
- Mix fixed (`px`) and flexible (`fr`) units.

🔹 4. Gap

css

```
gap: 20px;            /* shorthand for row and column gap */
row-gap: 10px;
column-gap: 20px;
```

Used to **control spacing** between grid items without margin.

🔹 5. Grid Lines & Placement

css
```
.item {
  grid-column: 2 / 4;
  grid-row: 1 / 3;
}
```

- Places the item from **column line 2 to 4**, **row line 1 to 3**.

🔹 6. Grid Areas (Named Layouts)

📝 HTML (`grid-area.html`)

html
```
<div class="layout">
  <header>Header</header>
  <nav>Menu</nav>
  <main>Main</main>
  <footer>Footer</footer>
</div>
```

🎨 CSS (`grid-area.css`)

css
```
.layout {
  display: grid;
  grid-template-areas:
    "header header"
    "nav main"
    "footer footer";
  grid-template-columns: 1fr 3fr;
  gap: 10px;
}

header { grid-area: header; }
nav    { grid-area: nav; }
```

```
main   { grid-area: main; }
footer { grid-area: footer; }

.layout > * {
  background-color: #e2e2e2;
  padding: 15px;
}
```

🔹 7. Implicit vs Explicit Grid

- **Explicit**: defined using `grid-template-columns/rows`
- **Implicit**: automatically added rows/columns using `grid-auto-rows` or `grid-auto-columns`

css

```
.grid-container {
  display: grid;
  grid-template-columns: 1fr 1fr;
  grid-auto-rows: 100px;
}
```

🔹 8. `grid-auto-flow`

css

```
.grid-container {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  grid-auto-flow: column;  /* default is row */
}
```

Controls how items fill the grid (row-wise or column-wise).

🔹 9. Nested Grids

html

```
<div class="outer-grid">
  <div class="inner-grid">
    <div>1</div><div>2</div><div>3</div>
  </div>
</div>
```

css

```
.outer-grid {
  display: grid;
  grid-template-columns: 1fr;
}

.inner-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}
```

🔹 10. Responsive Design with Grid

Use `media queries` with `repeat(auto-fit, minmax(...))`:

css

```
.grid-container {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 10px;
}
```

Automatically adjusts grid items based on screen width.

🔹 11. CSS Grid vs Flexbox

Feature	CSS Grid	Flexbox
Layout Direction	2D (rows + columns)	1D (row or column)
Use Case	Page layout, templates	Navbar, cards, buttons
Positioning	Grid lines, areas	Order, grow, shrink

✅ Summary Table

Property	Purpose
`display: grid`	Initiates grid layout
`grid-template-columns`	Defines columns
`grid-template-rows`	Defines rows
`grid-gap` / `gap`	Controls spacing
`grid-area`	Assigns named regions
`grid-column`, `grid-row`	Controls item placement
`grid-auto-flow`	Flow direction of items
`auto-fit`, `minmax`	Responsive grid settings